LogPulse - приём, разбор и статистика сервисных логов

Дисциплина: Внедрение и поддержка компьютерных систем **Форма:** индивидуальное практическое задание для ликвидации академической задолженности **Учебный год:** 2025/2026, 4 курс, группа 9/4-РПО-23/1 **Выполнил:** Гущин Даниил (GitHub: shargabong) **Преподаватель:** Макаров Максим Николаевич **Дата:** 08.11.2026

1. Цель

Нужно было сделать LogPulse - систему из HTTP API на FastAPI, воркера и Redis. Она принимает строки логов, проверяет их регулярками, валидные записи складывает в Redis Stream, воркер их читает и считает статистику. Всё поднимается одним docker compose.

2. Что сделано

Парсер и регулярки

Формат строки такой:

```
<UTC ISO-8601 Z> <LEVEL> <service> <IPv4> <METHOD> <path> <status> <N>ms rid=<8 hex>
```

Разбор делает одна регулярка с именованными группами (timestamp, level, service, ip, method, path, status, duration_ms, rid), строка проверяется целиком через re.fullmatch, без split. Отдельно сделал ещё несколько регулярок: TS_RE, IPV4_RE, RID_RE, PATH_RE и PROBLEM_RE (поиск WARN/ERROR). На каждую есть свой тест.

Потом поля проверяются по типам: timestamp через datetime, IP через ipaddress, код ответа должен попадать в 100-599, длительность в 0-600000 мс, а level/service/ method - в свои списки. Если что-то не сходится, функция кидает ParseError с причиной, и такая строка в очередь не попадает.

Тестовый файл data/sample.log: 45 корректных строк (три сервиса, три уровня, 12 разных кодов, уникальные rid) и 6 намеренно испорченных.

Таблица 1. Проверка регулярок и ошибок

Пример / тип	Ожидалось	Получилось	Пояснение
INFO auth 192.0.2.10 GET /api/login 200 12ms rid=00000001	Разбор	Разобрана, status=200 (int)	Строка совпала с маской
ERROR orders ... 503 742ms rid=ee44cc22	Разбор	Разобрана, попала в logpulse:slow	742 больше порога 500
2026-13-45T99:99:99Z INFO ...	Ошибка timestamp	timestamp не корректная дата	Маску прошла, datetime не пропустил
INFO auth 999.1.1.1 ...	Ошибка IP	неверный IP: 999.1.1.1	ipaddress не принял октет 999
DEBUG auth ...	Плохой level	строка не соответствует формату	DEBUG нет в маске
ERROR orders ... 999 ...	Плохой статус	код ответа вне 100-599: 999	Маску прошла, диапазон нет
INFO payment ... 200 -5ms	Отрицательная длительность	строка не соответствует формату	-5ms не совпало с \d+ms
WARN orders ... rid=ZZZ12cd3	Плохой rid	строка не соответствует формату	rid не из hex

Пример / тип	Ожидалось	Получилось	Пояснение
Пустая строка (пробелы)	Отказ	пустая строка	Проверка до маски

Redis: очередь и статистика

Очередь сделал через Redis Stream logpulse:events. API кладёт туда события командой XADD. Воркер читает их через XREADGROUP (группа logpulse-workers), обновляет статистику и подтверждает через XACK.

Таблица 2. Структуры Redis

Ключ	Тип	Запись / чтение	Зачем
logpulse:events	Stream	XADD / XREADGROUP, XREVRANGE	Очередь событий
logpulse:stats:level	Hash	HINCRBY / HGETALL	Сколько событий каждого уровня
logpulse:stats:service	Hash	HINCRBY / HGETALL	Сколько событий по сервисам
logpulse:stats:status	Hash	HINCRBY / HGETALL	Сколько ответов по кодам
logpulse:slow	Sorted Set	ZADD / ZREVRANGE	Рейтинг медленных (score = duration_ms, member = rid)

Порог медленного запроса: duration_ms >= 500.

"Принято в очередь" и "обработано воркером" - это разные вещи. Принято - это XLEN у потока, обработано - это total из /stats (сумма счётчиков уровней). После загрузки sample.log получается XLEN = 45 и total = 45.

FastAPI, Pydantic и ошибки

API на FastAPI: маршруты разбил по APIRouter, модели на Pydantic. Маршруты: /health, /ingest (принимает {"lines": [...]}), /stats и /events?limit=10. Лимиты: от 1 до 100 строк, каждая от 1 до 500 символов. Доступ к Redis идёт через Depends, в тестах он подменяется. Ошибки: 422 на неверный запрос, 503 если Redis недоступен, все ответы в одном формате {"error": {"type", "message", "details"}}.

Таблица 3. Проверка API и тестов

Сценарий	Вход	Ожидалось	Получилось
POST /ingest	3 валидные + 2 ошибки	accepted=3, rejected=2	Так и есть, 2 пояснения
POST /ingest	sample.log (45 + 6)	accepted=45, rejected=6	Подтверждено
GET /stats	после воркера	Агрегаты из Redis	total=45, уровни 15/15/15
GET /health	Redis работает	Готовность	200, {"status":"ok","redis":"ok"}
GET /health	Redis остановлен	503	503, быстро
GET /events?limit=1	2 события	1 последнее	count=1, rid=ee44cc22
POST /ingest	101 строка	Отказ по лимиту	422
pytest	тест парсера, IP, top_slow	Пройдены	Пройдены

Docker Compose

Есть Dockerfile, compose.yaml и .env.example. В compose три сервиса: api, worker, redis. Настройки идут через переменные окружения, внутри контейнеров Redis доступен по имени redis, а не по localhost. У redis есть healthcheck, api и worker ждут его готовности. Данные Redis лежат в volume redis-data. HTTP открыт только на 127.0.0.1, порт redis наружу не торчит.

Таблица 4. Журнал инцидента

Шаг	Действие	Ожидалось	Факт	Доказательство
1	docker compose up --build -d	Три контейнера готовы	redis healthy, api и worker подняты	вывод docker compose ps
2	Загрузить sample.log	Статистика построена	accepted=45, rejected=6, total=45	ответ /ingest и /stats
3	docker compose restart redis	Краткая недоступность	Воркер ждёт и повторяет попытку	строки лога воркера о повторе
4	Дождаться восстановления	Сервис снова работает	/health = 200, поток дообработан	ответ /health, XINFO GROUPS (lag=0)
5	Повторить /stats	Данные на месте	total=45, не изменилось	ответ /stats после restart

Отдельно проверил полную остановку redis: `/health`, `/stats` и `/ingest` возвращают 503, а после `docker compose start redis` доступность и статистика поднимаются обратно из volume.

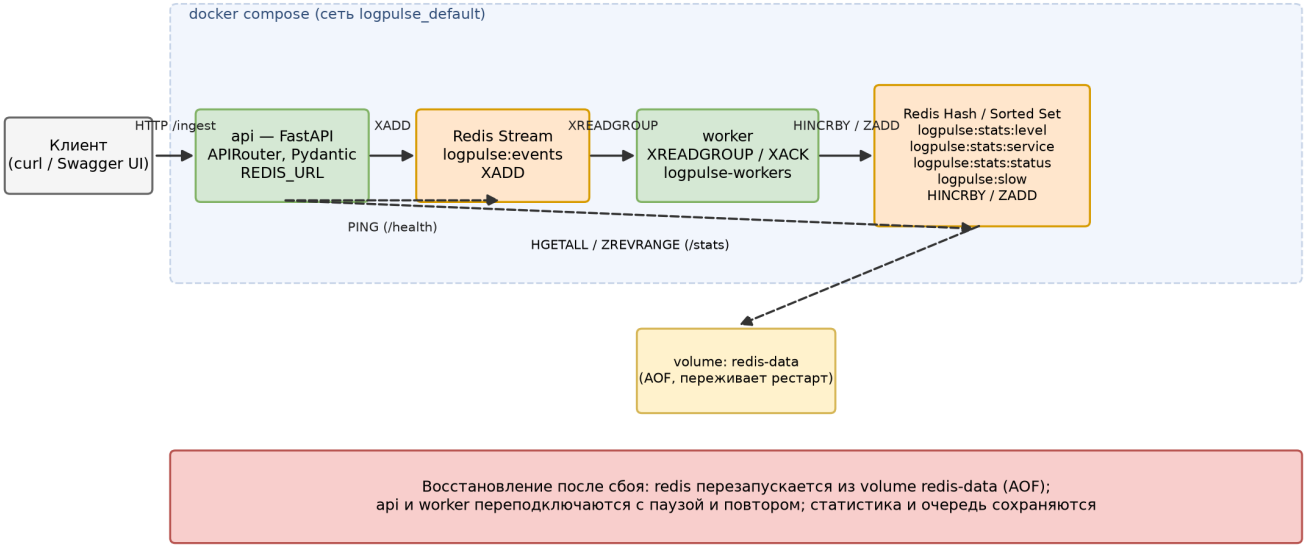
Git и тесты

Тестов получилось 36 (нужно было минимум 12). Покрыл: корректную строку, битый timestamp, плохой IP, неподдерживаемый уровень, плохой статус, отрицательную и слишком большую длительность, плохой rid, пустую строку, накопление счётчиков, рейтинг медленных, лимиты /events и /ingest, а также /health без Redis (503).

Репозиторий вёл с самого начала: получилось 15 коммитов, разработка в двух ветках (feature/parser и feature/redis) со слиянием в main, плюс .gitignore. История выгружена в history.bundle.

```
$ git log --graph --oneline --all --decorate
* bda33c2 (HEAD -> main) убрал лишнее, номер строки в парсере
* a60b4e3 перечитал отчёт, поправил под код
* 8d56fd4 упростил код, оставил синхронный redis
* 6e982ab поправил отчёт
* e0102d4 добавил .dockerignore
* 6bfabfc ридми, схема и отчёт
*   cfbc913 слил redis и воркер в main
|\
| * 657b687 (feature/redis) воркер читает стрим и считает статистику
| * a8dc261 сделал апи: health, ingest, stats, events
| * e93d1c2 модели и работа со статистикой в redis
* |   7254ec9 слил парсер в main
|\ \
| |/
|/|
| * f2160bc (feature/parser) накидал тестовых логов и проверку
| * 097db3e сделал парсер логов, регулярка и поля
|/
* fb97d9b начал проект, накидал структуру
```

3. Архитектура



Путь события: Клиент -> API -> Redis Stream -> worker -> Redis Hash/Sorted Set. Отдельной стрелкой показал, что API читает статистику из Redis. На схеме ещё контейнеры, переменные окружения (REDIS_URL), volume redis-data и восстановление после перезапуска. Редактируемый исходник: architecture.drawio.

4. Примеры логов и ответов


```
{
  "accepted": 3,
  "rejected": 2,
  "errors": [
    {"line": 4, "reason": "код ответа вне 100-599: 999"},
    {"line": 5, "reason": "неверный IP: 999.1.1.1"}
  ]
}
```

```
{
  "total": 45,
  "by_level": {"INFO": 15, "WARN": 15, "ERROR": 15},
  "by_service": {"auth": 15, "orders": 15, "payment": 15},
  "by_status": {"200": 4, "201": 4, "204": 4, "301": 4, "400": 4, "401": 4,
                "403": 4, "404": 4, "429": 4, "500": 3, "502": 3, "503": 3},
  "top_slow": [
    {"rid": "0000002c", "duration_ms": 1500},
    {"rid": "00000021", "duration_ms": 1500},
    {"rid": "00000016", "duration_ms": 1500},
    {"rid": "0000000b", "duration_ms": 1500},
    {"rid": "0000002b", "duration_ms": 980}
  ]
}
```


5. Скриншоты

LogPulse 1.0 OAS 3.1

/openapi.json

Приём и разбор сервисных логов, статистика в Redis.

default

GET /health Health

POST /ingest Ingest

GET /stats Stats

GET /events Events

Schemas

ErrorBody › Expand all object

ErrorResponse › Expand all object

EventItem › Expand all object

EventsResponse › Expand all object

HTTPValidationError › Expand all object

HealthResponse › Expand all object

IngestRequest › Expand all object

IngestResponse › Expand all object

ParseErrorItem › Expand all object

SlowRequest › Expand all object

StatsResponse › Expand all object

ValidationError › Expand all object

LogPulse 1.0 OAS 3.1

/openapi.json

Приём и разбор сервисных логов, статистика в Redis.

default

GET `/health` Health

POST `/ingest` Ingest

Принять пакет строк: валидные уходят в Redis Stream, ошибочные в errors.

Parameters | Cancel | Reset

No parameters

Request body required — application/json

Edit Value | Schema

```
{
  "lines": [
    "2026-10-20T12:34:56Z INFO auth 192.0.2.10 POST /api/login 200 43ms rid=ab12cd34",
    "2026-10-20T12:35:01Z ERROR orders 198.51.100.7 GET /api/orders/42 503 742ms rid=ee44cc22"
  ]
}
```

Execute | Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8080/ingest' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "lines": [
    "2026-10-20T12:34:56Z INFO auth 192.0.2.10 POST /api/login 200 43ms rid=ab12cd34",
    "2026-10-20T12:35:01Z ERROR orders 198.51.100.7 GET /api/orders/42 503 742ms rid=ee44cc22"
  ]
}'
```

Request URL

```
http://127.0.0.1:8080/ingest
```

Server response

Code	Details
200	Response body

```
{
  "accepted": 2,
  "rejected": 0,
  "errors": []
}
```

Download

Response headers

```
content-length: 39
content-type: application/json
date: Sun,09 Nov 2026 13:52:24 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	*No links*

Media type: application/json

Controls Accept header.

Example Value | Schema

```
{
  "accepted": 0,
  "rejected": 0,
  "errors": [
    {
      "line": 0,
      "reason": "string"
    }
  ]
}
```

422	Validation Error	*No links*

Media type: application/json

Example Value | Schema

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string"
    }
  ]
}
```

503	Redis недоступен	*No links*

Media type: application/json

Example Value | Schema

```
{
  "error": {
    "type": "string",
    "message": "string",
    "details": [
      {}
    ]
  }
}
```

GET `/stats` Stats

GET `/events` Events

Schemas

- ErrorBody — Expand all object
- ErrorResponse — Expand all object
- EventItem — Expand all object
- EventsResponse — Expand all object
- HTTPValidationError — Expand all object
- HealthResponse — Expand all object
- IngestRequest — Expand all object
- IngestResponse — Expand all object
- ParseErrorItem — Expand all object
- SlowRequest — Expand all object
- StatsResponse — Expand all object
- ValidationError — Expand all object

MINGW64:/f/dolgi/logpulse

```
$ docker compose exec -T redis redis-cli XLEN logpulse:events
45

$ docker compose exec -T redis redis-cli XINFO GROUPS logpulse:events
name
logpulse-workers
consumers
1
pending
0
last-delivered-id
1794146187038-2
entries-read
45
lag
0

$ docker compose exec -T redis redis-cli HGETALL logpulse:stats:level
INFO
15
WARN
15
ERROR
15

$ docker compose exec -T redis redis-cli HGETALL logpulse:stats:service
auth
15
orders
15
payment
15

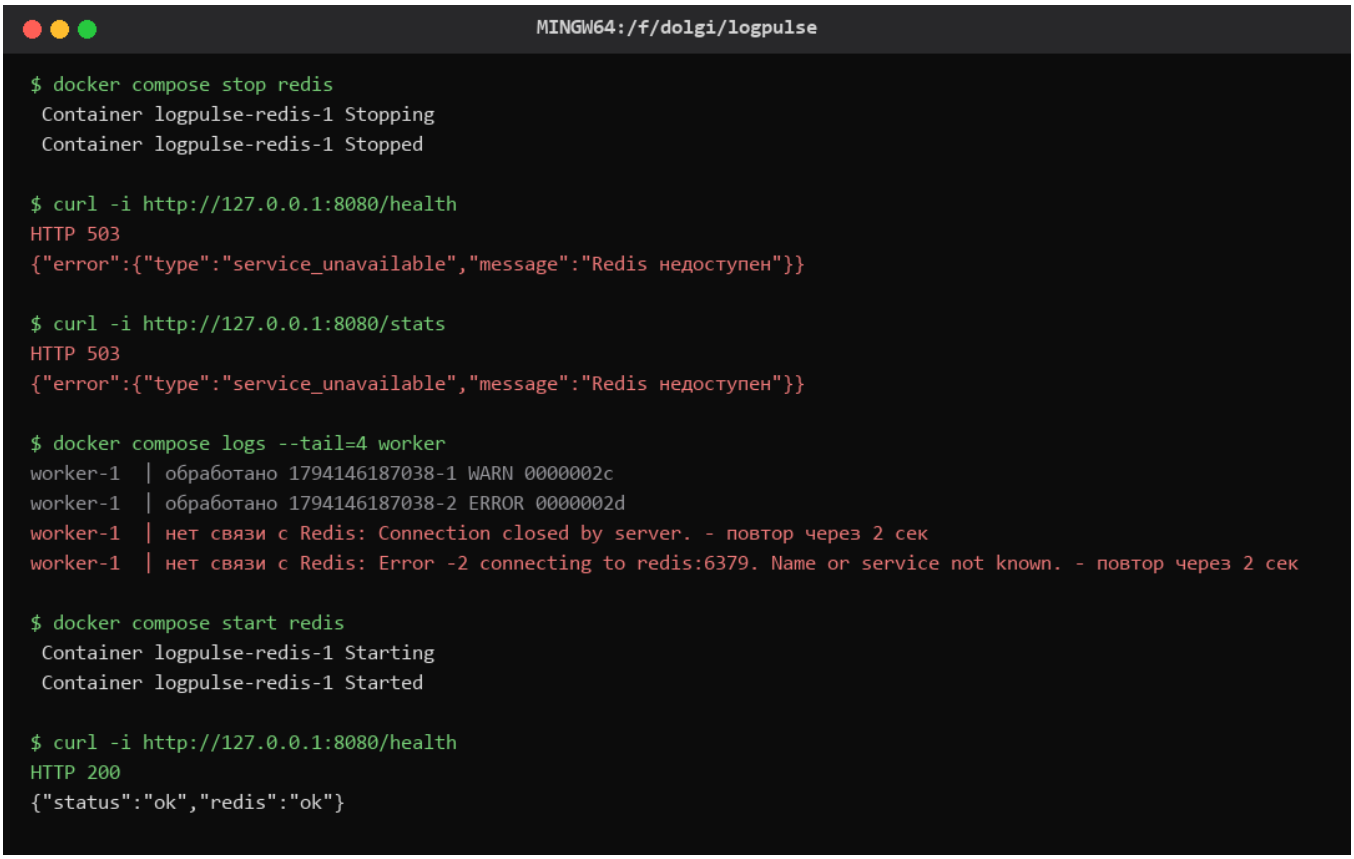
$ docker compose exec -T redis redis-cli HGETALL logpulse:stats:status
200
4
201
4
204
4
301
4
400
4
401
4
403
4
404
4
429
4
500
3
502
3
503
3

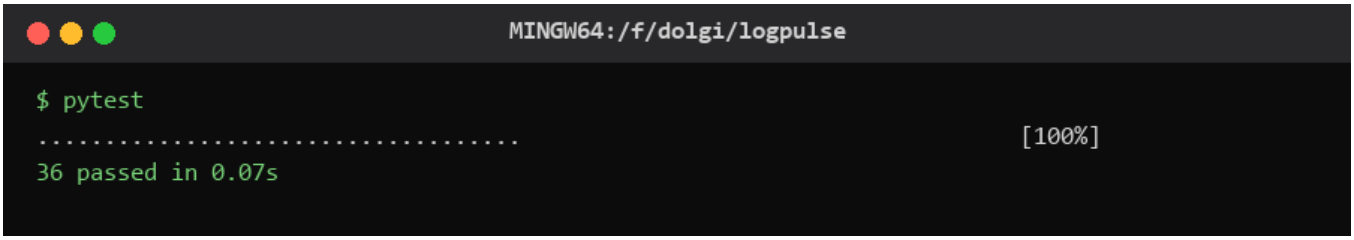
$ docker compose exec -T redis redis-cli ZREVRANGE logpulse:slow 0 4 WITHSCORES
0000002c
1500
00000021
1500
00000016
1500
0000000b
1500
0000002b
980
```

MINGW64:/f/dolgi/logpulse

```
$ docker compose ps
NAME                IMAGE               COMMAND                  SERVICE   CREATED        STATUS                        PORTS
logpulse-api-1      logpulse-api        "uvicorn app.main:ap…"   api       21 hours ago   Up About an hour (healthy)    127.0.0.1:8080->8000/tcp
logpulse-redis-1    redis:7-alpine      "docker-entrypoint.s…"   redis     21 hours ago   Up About an hour (healthy)    6379/tcp
logpulse-worker-1   logpulse-worker     "python worker.py"       worker    21 hours ago   Up 16 minutes                 8000/tcp

$ docker compose logs --tail=14 api worker redis
redis-1   | 370:C 08 Nov 2026 13:04:43.064 * Fork CoW for RDB: current 0 MB, peak 0 MB, average 0 MB
redis-1   | 1:M 08 Nov 2026 13:04:43.159 * Background saving terminated with success
redis-1   | 1:M 08 Nov 2026 13:40:55.065 * DB saved on disk
redis-1   | 1:M 08 Nov 2026 13:45:56.003 * 100 changes in 300 seconds. Saving...
worker-1  | обработано 1794146187036-0 WARN 00000020
redis-1   | 1:M 08 Nov 2026 13:45:56.003 * Background saving started by pid 3385
redis-1   | 3385:C 08 Nov 2026 13:45:56.011 * DB saved on disk
api-1     | INFO:     127.0.0.1:49322 - "GET /health HTTP/1.1" 200 OK
redis-1   | 3385:C 08 Nov 2026 13:45:56.011 * Fork CoW for RDB: current 0 MB, peak 0 MB, average 0 MB
redis-1   | 1:M 08 Nov 2026 13:45:56.104 * Background saving terminated with success
redis-1   | 1:M 08 Nov 2026 13:56:19.169 * DB saved on disk
api-1     | INFO:     127.0.0.1:58124 - "GET /health HTTP/1.1" 200 OK
api-1     | INFO:     127.0.0.1:38784 - "GET /health HTTP/1.1" 200 OK
api-1     | INFO:     127.0.0.1:47088 - "GET /health HTTP/1.1" 200 OK
api-1     | INFO:     127.0.0.1:47708 - "GET /health HTTP/1.1" 200 OK
worker-1  | обработано 1794146187036-1 ERROR 00000021
worker-1  | обработано 1794146187036-2 INFO 00000022
worker-1  | обработано 1794146187036-3 WARN 00000023
api-1     | INFO:     127.0.0.1:48178 - "GET /health HTTP/1.1" 200 OK
worker-1  | обработано 1794146187036-4 ERROR 00000024
worker-1  | обработано 1794146187037-0 INFO 00000025
worker-1  | обработано 1794146187037-1 WARN 00000026
worker-1  | обработано 1794146187037-2 ERROR 00000027
api-1     | INFO:     127.0.0.1:36030 - "GET /health HTTP/1.1" 200 OK
redis-1   | 1:M 08 Nov 2026 14:01:20.081 * 100 changes in 300 seconds. Saving...
redis-1   | 1:M 08 Nov 2026 14:01:20.081 * Background saving started by pid 4527
api-1     | INFO:     127.0.0.1:36520 - "GET /health HTTP/1.1" 200 OK
api-1     | INFO:     127.0.0.1:36864 - "GET /health HTTP/1.1" 200 OK
api-1     | INFO:     127.0.0.1:35556 - "GET /health HTTP/1.1" 200 OK
api-1     | INFO:     127.0.0.1:33158 - "GET /health HTTP/1.1" 200 OK
api-1     | INFO:     127.0.0.1:57254 - "GET /health HTTP/1.1" 200 OK
redis-1   | 4527:C 08 Nov 2026 14:01:20.087 * DB saved on disk
redis-1   | 4527:C 08 Nov 2026 14:01:20.088 * Fork CoW for RDB: current 0 MB, peak 0 MB, average 0 MB
redis-1   | 1:M 08 Nov 2026 14:01:20.182 * Background saving terminated with success
worker-1  | обработано 1794146187037-3 INFO 00000028
api-1     | INFO:     127.0.0.1:59628 - "GET /health HTTP/1.1" 200 OK
worker-1  | обработано 1794146187037-4 WARN 00000029
worker-1  | обработано 1794146187037-5 ERROR 0000002a
worker-1  | обработано 1794146187038-0 INFO 0000002b
api-1     | INFO:     127.0.0.1:50038 - "GET /health HTTP/1.1" 200 OK
worker-1  | обработано 1794146187038-1 WARN 0000002c
worker-1  | обработано 1794146187038-2 ERROR 0000002d
```





6. Инцидент с Redis

Проверял сценарий из задания: запуск, загрузка sample.log, docker compose restart redis, ожидание и повторный /health и /stats. Дополнительно решил проверить полную остановку redis, а не только restart.

Наткнулся на проблему: когда Redis лежит, /health не отдавал 503, а просто висел. Сначала думал, что завис сам uvicorn. Оказалось, у клиента redis не был задан таймаут сокета, поэтому запрос ждал ответа бесконечно. Добавил socket_connect_timeout и socket_timeout по 2 секунды, теперь /health и /stats быстро отдают 503.

Данные при перезапуске не теряются. Redis пишет с --appendonly yes в volume redis-data. После restart и даже после stop/start total=45, счётчики и top_slow остались на месте. Команду down -v для такой проверки использовать нельзя, она удаляет volume.

При недоступном Redis приложение не падает: uvicorn продолжает работать, все маршруты отдают 503 в общем формате, а как только Redis вернулся - снова 200. Воркер тоже не падает: пишет в лог, что нет связи, ждёт 2 секунды и пробует снова, после восстановления читает с того же места группы.

Про полную остановку: когда redis реально stop, а не restart, из DNS пропадает имя сервиса, и в логе воркера появляется запись вида "нет связи с Redis: Error -2 connecting to redis:6379. Name or service not known". Воркер её ловит, ждёт 2 секунды и пробует снова, на работу это не влияет. При обычном restart такого нет, потому что имя сервиса остаётся.

В итоге: сервис нормально переживает недоступность Redis, данные сохраняются, воркер переподключается. Баг с зависанием /health был настоящий, поправил.

7. Комплектность

Таблица 5. Контроль комплектности

Артефакт	Требование	Файл	Проверено
FastAPI-проект	FastAPI + парсер + worker	app/, worker.py	Да
Redis	Stream + Hash + Sorted Set	app/stats.py, app/config.py	Да
Docker Compose	3 сервиса + volume	compose.yaml, Dockerfile	Да
Тесты	>= 12 pytest	tests/ (36 тестов)	Да
Данные	>= 40 логов и >= 5 битых	data/sample.log (45 + 6)	Да
Git	>= 6 коммитов, 2 ветки	history.bundle	Да
Отчёт	PDF + 5 таблиц + скриншоты	report.pdf	Да
Схема	.drawio	architecture.drawio	Да
Домашние работы	Все прошлые ДЗ	сдаются отдельно	Да

8. Ограничения

- Если сообщение придёт воркеру дважды, счётчики могут увеличиться дважды. В задании такие дубликаты описаны как допустимое ограничение, так что оставил как есть.
- Воркер один, шардирования нет.